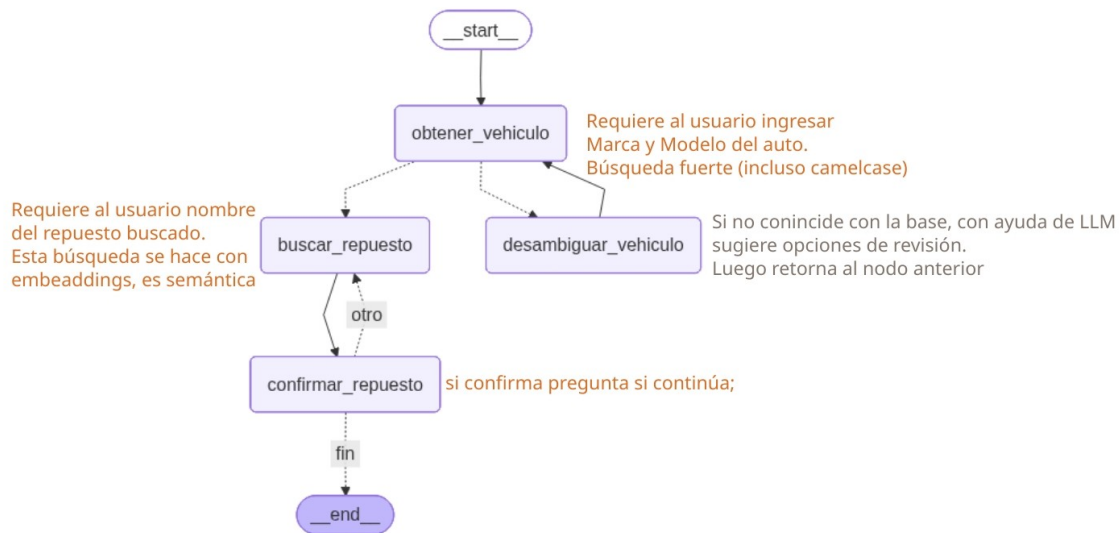


Documentación del flujo LangGraph – Proyecto Repuestos

Este documento describe la estructura del flujo implementado en el cuaderno "implementacion_01.ipynb". Se detallan el modelo de estado, las funciones auxiliares, los nodos del grafo de LangGraph y las aristas (transiciones) entre ellos.



1. Modelo de estado

El proyecto define varios TypedDict que modelan la información que circula por el grafo:

1.1. Vehiculo

TypedDict con los campos:

- marca: nombre de la marca del vehículo (str u opcional).
- modelo: modelo del vehículo (str u opcional).

Se utiliza dentro del State como representación compacta del vehículo sobre el cual se buscan repuestos.

1.2. ItemRepuesto

TypedDict que representa un repuesto concreto seleccionado para el pedido.

Campos principales:

- descripcion: descripción comercial del repuesto.

- `marca_repuesto`: marca del repuesto.
- `marca_vehiculo`: marca del vehículo al que aplica.
- `modelo`: modelo de vehículo al que aplica.
- `anio`: rango o año de aplicación.
- `precio`: precio unitario.
- `stock`: stock disponible.
- `score`: puntaje de relevancia devuelto por la búsqueda semántica.

1.3. State

El State es un TypedDict que agrupa la información compartida por todos los nodos del grafo. Los campos relevantes en esta versión son:

- `vehiculo`: objeto Vehiculo con la marca y el modelo actualmente seleccionados.
- `items`: lista de ItemRepuesto que representan los ítems agregados al pedido.
- `descripcion_actual`: texto libre que describe el repuesto que el usuario está buscando o el mensaje más reciente.
- `candidatos`: lista de repuestos candidatos devueltos por la búsqueda semántica.
- `marca_intento` / `modelo_intento`: último intento de marca y modelo ingresado por el usuario, usado para desambiguar.
- `sigue_buscando`: bandera booleana que indica si el usuario desea seguir agregando repuestos.

2. Funciones auxiliares

2.1. `validar_marca_modelo(marca, modelo, coleccion)`

Consulta la colección MongoDB para verificar que:

- la marca exista en el campo "Marca Vehículo"; y
- el modelo exista en el campo "Modelo".

Si alguno de los dos no está presente, devuelve False. Si ambos existen, devuelve True. Es la primera capa de validación estructural antes de avanzar en el flujo.

2.2. `buscar_semantico_filtrado(marca_vehiculo, modelo, descripcion, n)`

Realiza una búsqueda vectorial en MongoDB combinando una consulta semántica (por texto) con filtros por marca de vehículo y modelo. Utiliza el campo vectorial para calcular un score de similitud y devuelve una lista de documentos candidatos con su score asociado.

La función encapsula el pipeline de agregación con etapas como:

- búsqueda vectorial (`vectorSearch`);

- proyección de campos (Descripción, Marca, Marca Vehículo, Modelo, Año, Precio, Stock, score);
- limit n para recortar la cantidad de resultados.

2.3. **filtrar_por_score_relativo(resultados, min_abs=0.6, factor_rel=0.9)**

Recibe una lista de resultados con un campo numérico 'score' y aplica un doble filtro:

- descarta los resultados cuyo score absoluto sea menor que min_abs;
- descarta los resultados cuyo score sea menor que factor_rel multiplicado por el mejor score de la lista.

Devuelve solo los candidatos suficientemente buenos, manteniendo un conjunto pequeño y relevante de repuestos.

2.4. **modelos_similares(marca, modelo, coleccion)**

Dada una marca y un modelo ingresados por el usuario, intenta encontrar vehículos similares en la base. El procedimiento es:

- 1) Normalizar marca y modelo como cadenas.
- 2) Construir filtros con \$regex case-insensitive escapando el texto del usuario.
- 3) Buscar primero por coincidencia de marca y modelo.
- 4) Si no hay resultados, buscar solo por marca.
- 5) Si aún no hay, buscar solo por modelo.

Devuelve una lista de documentos con 'Marca Vehículo', 'Modelo' y 'Año', que luego se usan para que el LLM genere un mensaje de desambiguación amable.

3. **Nodos del grafo**

Los nodos son funciones que reciben un State y devuelven un nuevo State (o una actualización del mismo). Cada nodo representa un paso lógico en la conversación con el usuario.

3.1. **nodo_obtener_vehiculo(state)**

Responsabilidad: pedir al usuario la marca y el modelo del vehículo.

Flujo interno:

- Solicita por consola (input) la marca y el modelo.
- Guarda esos valores en 'marca_intento' y 'modelo_intento' dentro del State, de modo que puedan ser usados si luego hay que desambiguar la selección.
- Valida la combinación llamando a validar_marca_modelo(...).

- Si la validación falla, imprime un mensaje de error, deja 'vehiculo' como None y devuelve el State actualizado con los intentos. Esto hará que el router envíe el control al nodo de desambiguación.
 - Si la validación es exitosa, imprime un mensaje de éxito y actualiza 'vehiculo' con un dict {'marca': ..., 'modelo': ...}.
- En ambos casos devuelve el State extendido con la información del intento.

3.2. nodo_desambiguar_vehiculo(state)

Responsabilidad: ayudar al usuario cuando la combinación marca/modelo no coincide exactamente con la base.

Flujo interno:

- Toma 'marca_intento' y 'modelo_intento' del State.
- Llama a modelos_similares(...) para obtener una lista de vehículos cercanos a lo que el usuario escribió.
- Construye un prompt que incluye la marca y modelo ingresados, y la lista de sugerencias encontradas.
- Llama al LLM (ChatGroq) para generar un mensaje amable que explique el problema y presente opciones similares.
- Imprime el mensaje generado y lo guarda en 'descripcion_actual' para que quede disponible en el State.
- Mantiene 'vehiculo' en None, de manera que el flujo vuelva a nodo_obtener_vehiculo para un nuevo intento.

3.3. nodo_buscar_repuesto(state)

Responsabilidad: a partir del vehículo seleccionado, solicitar al usuario qué repuesto necesita y buscar candidatos en la base de datos.

Flujo interno:

- Recupera 'vehiculo' del State y extrae 'marca' y 'modelo'. Si aún no están definidos, muestra un mensaje de error y devuelve el State sin cambios.
- Pide al usuario una descripción libre del repuesto (texto).
- Llama a buscar_semantico_filtrado(...) con la marca, modelo y descripción, obteniendo una lista de resultados con score.
- Aplica filtrar_por_score_relativo(...) para quedarse solo con los mejores candidatos.
- Imprime en consola las opciones de repuestos sugeridos, con descripción, marca, vehículo, año, precio, stock y score.

- Actualiza 'descripcion_actual' con la descripción ingresada y 'candidatos' con la lista filtrada de repuestos.

El nodo no decide si el usuario agrega o no un repuesto: solo prepara el terreno para la confirmación.

3.4. nodo_confirmar_repuesto(state)

Responsabilidad: mostrar los candidatos de repuesto al usuario, permitirle elegir uno y agregarlo a la lista de items.

Flujo interno:

- Toma la lista de 'candidatos' del State. Si está vacía, informa que no hay opciones y devuelve el State sin cambios.
- Vuelve a imprimir las opciones disponibles, numeradas.
- Pide al usuario que elija un número de candidato.
- A partir de esa elección, construye un ItemRepuesto con los campos relevantes (descripción, marca_repuesto, marca_vehiculo, modelo, año, precio, stock, score) y lo agrega a la lista 'items' del State.
- Pregunta al usuario si quiere seguir buscando más repuestos (por ejemplo, con una entrada 's' / 'n').
- Actualiza 'sigue_buscando' en el State con un booleano (True si quiere seguir, False en caso contrario).
- Limpia 'candidatos' y 'descripcion_actual' dado que la interacción sobre ese repuesto se cerró.

De esta forma el nodo centraliza la lógica de confirmación y actualización de la lista de ítems del pedido.

3.5. ex_nodo_obtener_vehiculo(state)

Versión anterior del nodo de obtención de vehículo, conservada a modo histórico. Su lógica era similar a la del nodo_obtener_vehiculo actual, pero sin el soporte para guardar marca_intento/modelo_intento ni la integración con el nodo de desambiguación. Puede servir como referencia de la primera iteración del flujo.

4. Routers y funciones de transición

Además de los nodos, el grafo define funciones de transición que devuelven el nombre simbólico de la rama que se debe tomar a continuación.

4.1. valida_vehiculo_completo(state) -> str

Se utiliza como función condicional después del nodo 'obtener_vehiculo'. Examina el campo 'vehiculo' del State:

- Si existe y tiene 'marca' y 'modelo' no nulos, devuelve la cadena 'buscar_repuesto'.
- En caso contrario, devuelve 'desambiguar_vehiculo'.

Esta función determina si el flujo puede avanzar a la búsqueda de repuestos o si necesita antes pasar por el nodo de desambiguación para ayudar al usuario a corregir su selección de vehículo.

4.2. **transicion_confirmar_repuesto(state) -> str**

Router que se usa después del nodo 'confirmar_repuesto'. Su lógica es sencilla:

- Si 'sigue_buscando' es True, devuelve 'otro'.
- Si 'sigue_buscando' es False o no está seteado, devuelve 'fin'.

De esta manera, el grafo puede decidir si vuelve a 'buscar_repuesto' para que el usuario agregue más ítems al pedido, o si cierra el flujo y termina en END.

5. **Aristas del grafo**

El grafo se construye con StateGraph(State) y se pueblan nodos y aristas. A nivel conceptual, las transiciones son las siguientes:

5.1. **Arista inicial**

- START → obtener_vehiculo

El flujo siempre comienza pidiendo al usuario que indique la marca y el modelo de su vehículo.

5.2. **Arista condicional desde 'obtener_vehiculo'**

Se usa graph.add_conditional_edges("obtener_vehiculo", valida_veiculo_completo, {...}). Las ramas son:

- "buscar_repuesto" → nodo 'buscar_repuesto' (si el vehículo está bien definido).
- "desambiguar_vehiculo" → nodo 'desambiguar_vehiculo' (si la combinación marca/modelo no es válida).

Esto implementa una bifurcación temprana en el flujo según la calidad de la selección de vehículo.

5.3. **Arista desde 'desambiguar_vehiculo' de vuelta a 'obtener_vehiculo'**

- desambiguar_vehiculo → obtener_vehiculo

Una vez que el LLM genera el mensaje de sugerencias, el grafo vuelve al nodo 'obtener_vehiculo' para que el usuario ingrese nuevamente marca y modelo (idealmente guiado por las opciones sugeridas).

5.4. Arista desde 'buscar_repuesto' hacia 'confirmar_repuesto'

- buscar_repuesto → confirmar_repuesto

Cuando el sistema ya tiene una lista de candidatos de repuesto para el vehículo indicado, pasa al nodo en el que el usuario elige cuál agregar al pedido.

5.5. Aristas condicionales desde 'confirmar_repuesto'

Se usa `graph.add_conditional_edges("confirmar_repuesto", transicion_confirmar_repuesto, {...})`, con dos ramas:

- "otro" → 'buscar_repuesto': el usuario desea seguir agregando repuestos al pedido.

- "fin" → END: el usuario no quiere agregar más ítems y el flujo concluye.

Este patrón permite un ciclo natural de búsqueda → confirmación → decidir si continuar o terminar.

6. Cierre

En conjunto, estos nodos, funciones auxiliares y aristas implementan un flujo conversacional para:

- 1) Establecer correctamente el vehículo del usuario (incluyendo ayuda con desambiguación usando un LLM).
- 2) Buscar repuestos relevantes mediante una combinación de búsqueda semántica y filtros por vehículo.
- 3) Permitir al usuario confirmar y agregar repuestos a un pedido, iterando tantas veces como sea necesario.

El diseño separa claramente las responsabilidades: validación de datos, búsqueda semántica, desambiguación asistida por LLM, y manejo del ciclo de confirmación de repuestos.

7. ToDo

- 1) la lista de repuestos no mira por fuera del stock interno; si no se encuentra el repuesto que se necesita debería ir para otro lado, buscarlo, ofrecerlo y sumarlo al listado posteriormente
- 2) en ninguna interacción con el usuario hay la opción de abortar
- 3) **Súper importante**: las salidas por pantalla son un horror... probablemente debería implementarse sobre un chat y listo... pero las impresiones del chat no serían directamente interacciones con LLM.